Fewer Executions, More Seconds:
A Pre-Registered Evaluation of LLM-Planned Concurrency Experiments for Services Backed by a Key-Value Store

Harthik Mallichetty
Team MetaMorphic
https://github.com/harthikrm/mylonite

November 2026

Preprint. Draft prepared after the Meta AI Developer Hackathon; not peer reviewed.

Abstract

Two request handlers that are each correct can lose data when they run at the same time against a shared Redis or Valkey key. Such store-mediated races involve no data race inside any process, so memory race detectors cannot see them, and tests that issue one request at a time never exercise them. We present *Mylonite*, a harness that places a delay-only proxy between an unmodified service and its store and controls the order in which the service's commands reach the store. A large language model, Meta's Muse Spark, reads the service's source and proposes experiments as schema-validated JSON: which requests to race, after what setup, and in which command order. Deterministic code runs the experiments and decides every verdict. A reported violation must replay 20 of 20 times with an identical trace hash, and must not occur on a corrected version of the service.

We ask whether the model's choice of experiment is worth its cost. A single developer ran a pre-registered study. The method was frozen before any held-out target existed. Two held-out defects were injected mechanically into newly written services, at sites selected by a public randomness beacon. The model was compared against four non-AI baselines, including a conflict-directed systematic search, on three metrics that were tested separately. With 20 trials per method per target, the model with source access needed a median of 1 execution to find each held-out bug, against 764 and 97 for the best baseline (Holm-adjusted exact Mann–Whitney $p = 8.7 \times 10^{-11}$ and 2.0×10^{-5}). It was nonetheless slower and more expensive. Its median find took 41.8 s and 59.9 s, against 1.0–6.5 s for systematic search, because one execution costs about 9 ms; it would be faster only if an execution took more than 0.055 s and 0.62 s respectively. Without the source, the model was not better than the conventional baselines as a group on any metric. In 200 negative-control trials there were no false alarms. The evaluation is small, with two lost-update defects and one developer, and we report the limitations and deviations recorded in the project.

1 Introduction

A lost update needs only two requests. Each reads a value from the store, computes on it, and writes it back, so the second write silently erases the first. When the shared state lives in an external key-value store such as Redis or Valkey, the race is between commands on separate network connections,

not between threads that share memory. ThreadSanitizer and the Go race detector see nothing, and unit tests, which typically run one request at a time against an in-memory fake, never interleave two requests. ACIDRain found such application-level races in 12 e-commerce applications but had to reproduce them by hand [30]. A survey of 67 Rails applications found application-level validations used far more often than database transactions [4]. A live instance of the pattern is issue #3824 in the OpenTelemetry Demo [25]. Two concurrent `AddItem` calls to the C# cart service each read the cart from Valkey and write the whole cart back, so one item disappears. The issue was still open when we last checked it on 1 November 2026.

Controlled concurrency testing is mature for threads inside one process. CHESS [24], PCT [6], dynamic partial-order reduction [1, 7], POS [34], greybox and periodical scheduling [31, 32] and Fray [16] all explore thread interleavings with replay. Two gaps remain for store-mediated races. First, a client-side thread scheduler cannot control the order of commands inside an external store. Second, these tools largely assume that the bug-triggering *input* is given; PCT states this assumption explicitly [6]. Choosing the workload remains open: which operations to race, with which arguments, after which setup.

A language model that reads source code might make that choice well. Recent work uses large language models (LLMs) to guide *thread interleavings* [8, 35]. We study the complementary role, in which the model chooses the *workload* and a deterministic scheduler explores the interleavings. Our question is not whether an LLM can find such a bug. It is whether the model's choice is worth its cost against strong conventional search, under the same execution budget and the same correctness contract. Evaluations of LLM-based testing are easy to get wrong. The model may have memorized public bugs, and the author's priors may shape the targets. Results may also be cherry-picked, and run-to-run variance may go unreported, as documented for fuzzing evaluations [14]. We therefore treat the evaluation design as part of the contribution.

Contributions.

1. **Mylonite**, a harness for store-mediated races in unmodified services (§4). A delay-only proxy for RESP2 and RESP3 gates data commands and only ever delays them, so every schedule it produces is reachable by ordinary network delay. A contract oracle in plain code decides verdicts. The model proposes experiments as validated data and never runs code or judges outcomes.

2. **A held-out protocol for a single developer** (§6.3). A file-hash manifest freezes the method before any held-out code exists. The base services are written afterwards. Defects are injected by a frozen mutation script at sites chosen by a public drand round [15], filtered for serial equivalence, and sealed outside the repository. The analysis is pre-registered: an intersection–union "stronger-baseline" rule, exact tests with Holm correction, and a break-even analysis that converts executions into time.

3. **A mixed, pre-registered result** (§8). On both held-out bugs, Muse with source access needed a median of 1 execution, against 764 and 97 for the strongest baseline. It was significantly worse on wall-clock time and on dollars, because executions are cheap. Without the source it was not better than the conventional group on any metric. In 200 control trials there were no false positives.

4. **Disclosure.** We report the deviations recorded in the project, including a late freeze, a reduced held-out scope and one restart after a harness defect. The code, trial data and analysis are open.

What this paper does not claim. We do not claim that LLM-guided testing generalizes beyond the two held-out defects. Both are lost updates in small Python services, and Muse's trials are near-replicates of one plan per target (§9). The development results, including the OpenTelemetry cart, were known while the rules were written. They are reported separately and never pooled with the held-out results.

2 Background and Related Work

Controlled and randomized concurrency testing. Systematic testers serialize a program's threads and enumerate or sample interleavings. Iterative context bounding showed that small preemption bounds suffice for many bugs [23]. CHESS found bugs that months of stress testing had missed [24]. PCT gives a per-run probability of at least $1/(nk^{d-1})$ of finding a depth-d bug [6]. DPOR and optimal DPOR give complete exploration with reduction [1, 7]. POS improves on PCT on standard suites [34]. RFF adds greybox feedback, and PERIOD schedules threads periodically [31, 32]. Thomson et al. found that controlled random scheduling is a strong baseline that exposes many benchmark bugs within its first few schedules [26]. That finding motivates our random baseline. Fray brings these policies to the JVM [16]. All of these control threads in one process. Mylonite instead controls the order of commands *between* a service and its store, at the network boundary, in any language.

Application-level races through a store. Lu et al. found that nearly all non-deadlock concurrency bugs are atomicity or order violations, and most involve at most two threads [18]. In Go, the built-in race detector caught only half of the reproduced non-blocking bugs [27]. Bugs whose shared state is in a database or cache are invisible to memory-level detectors by construction. ACIDRain verified 22 attacks in 12 applications. It reproduced them manually, and some needed an injected proxy delay [30]. Feral concurrency control documents how often applications enforce integrity in application code rather than in the store [4]. Latency-injecting proxies delay traffic, but they neither choose orders nor check outcomes. Mylonite makes the proxy a systematic scheduler with an oracle.

Oracles. Linearizability [10] and serial-equivalence checking without a written specification [5] inform two serial checks that we do use. Every base service must pass 150 random serial workloads, and every mutant must be serially equivalent to its base (§6.3). We adopt Jepsen's history semantics, in which an operation is invoked and then completes as `ok`, `fail` or the indeterminate `info` [12]. Elle infers isolation anomalies from histories [13]. Mylonite's oracle is a small set of contract invariants interpreted by fixed code. A serial-equivalence oracle for operations declared atomic was designed but not implemented.

LLMs in testing and concurrency. REINS has an LLM agent produce event-order and state constraints for a controlled C/C++ scheduler, and reports new real-world bugs [35]. Fray exposes per-step scheduling to an LLM through an MCP server [8]. In both, the model steers interleavings and the test inputs are given. Learned input selection helped kernel concurrency testing in Snowcat [9]. LLM-guided fuzzers such as ChatAFL and Fuzz4All evaluate with repeated trials and effect sizes [21, 33]. Studies of LLM vulnerability detection report non-deterministic answers and high false-discovery rates [17, 28]. These results are why we keep the model out of every verdict.

Evaluation methodology. We follow the recommendations of Klees et al. for fuzzing evaluations (repeated trials, strong baselines, statistical tests, all results reported) [14], and of Arcuri and Briand for randomized algorithms [3]. We use the exact Mann–Whitney test [20], the Vargha–Delaney effect size $\hat{A}_{12}$ [29] and Holm's correction [11]. To our knowledge, an evaluation of LLM-guided testing in which a single developer cannot choose which bugs exist or where they are is uncommon. Our protocol is one attempt at it.

3 Problem Setting

Services, stores and workloads. A *target* is a service that keeps shared state in a key-value store speaking RESP (Valkey or Redis). It exposes a *catalog* of operations, each with typed arguments drawn from pre-registered *value domains*. A *workload* is up to 6 sequential setup calls, then one *concurrent round* of 2–4 calls, then observation calls that read the final state through the service's own API. Valkey executes commands one at a time, so the order of commands at the store is the whole store-level interleaving. An *execution* runs one workload under one schedule from a fresh reset. The unit of every budget is the *search execution*: a concurrent execution, or a profiling or exploration run that a method requests for itself.

Contracts and verdicts. Each target carries an owner-approved *contract*. It consists of invariants of a fixed set of kinds interpreted by our code, such as `counter_conserved` (the final value equals the initial value plus the amounts of successful increments), `non_negative` and `item_quantity_conserved`. The oracle returns `PASS` or `VIOLATION(clause)`. The executor marks an execution `INCONCLUSIVE` on a timeout, a harness error or a soundness failure. The design also provided for groups of operations that must be serializable, and for an `OBSERVATION` verdict when an outcome matches no serial order but no clause covers it. Neither was implemented, and every target's set of such groups is empty.

The question. For a method m, E_1 is the number of search executions up to and including the one that produced its first *verified* violation, right-censored at the execution budget; $E_1 = 1$ means the first execution. Our co-primary metrics are E_1 and the cost of reaching the first verified violation, in method-seconds and in dollars. The model's latency and price count toward its costs.

Running example. Listing 1 shows the `award` handler of a development fixture (D2). Its contract says that a member's balance equals the sum of the points of all successful awards. If two awards both run `GET` before either runs `SET`, one award is lost.

Listing 1: The development fixture D2 (`fixtures/dev/d2_counter/service.py`). The corrected version replaces the three body lines with one atomic `INCRBY`.

```
def award(r, member, points):
    balance = int(r.get(f"points:{member}") or 0)
    balance += points
    r.set(f"points:{member}", balance)
    return balance
```

4 Mylonite

Figure 1 shows the components. Every method, the model and the baselines alike, produces experiments in one shared search space. The same trial runner charges them against the same budgets, the same executor, proxy and oracle run them, and the same verifier checks them.

4.1 Delay-only proxy and scheduler

The proxy frames RESP2 and RESP3 and forwards the original bytes unchanged. It gates data commands on keys (for example `GET`, `SET`, `HGET`, `HSET`, `HINCRBY`, `INCRBY`, `DECRBY`). It schedules `MULTI`…`EXEC`, `WATCH`…`EXEC` and `EVAL` blocks as single units, and passes connection and administrative commands through. It obeys five rules:

1. It never drops, rewrites, duplicates or invents a command.
2. On each connection, commands execute and replies return in arrival order. The scheduler can only delay the head of a connection's queue.
3. Across connections, the scheduler chooses freely which head executes next.
4. At most one gated command or block is in flight to the store at a time, so the store's execution order equals the decision order.
5. Admission is a decision: the scheduler chooses when each concurrent request is issued.

A *decision point* occurs when no operation is computing inside the service. The scheduler then either admits a pending operation or executes the head command of a connection whose operation is waiting. Each decision lets at most one operation run, so every new command is attributed to that operation.

Soundness. TCP connections are independent, so network delay alone can produce any cross-connection order that respects per-connection order. Every schedule the proxy produces is therefore reachable in production, and a violation is real behavior, not an artifact of instrumentation. The converse does not hold. On a multiplexed client, where requests share one connection, in-process timing can produce orders the proxy cannot. This is a false-negative risk, and we disclose it. A soundness checker runs on every trace. On each connection it checks that every executed command had arrived, and it flags commands it cannot attribute or that bypassed gating. A lock enforces the rule of one command in flight. A failing trace is a harness error.

Policies. All methods use the same policies. `passthrough` applies no gating, `random(seed)` picks uniformly among legal choices, and `exhaustive` enumerates every decision sequence, admissions included, depth-first. `hint`→`exhaustive` follows a suggested order of store commands where it is

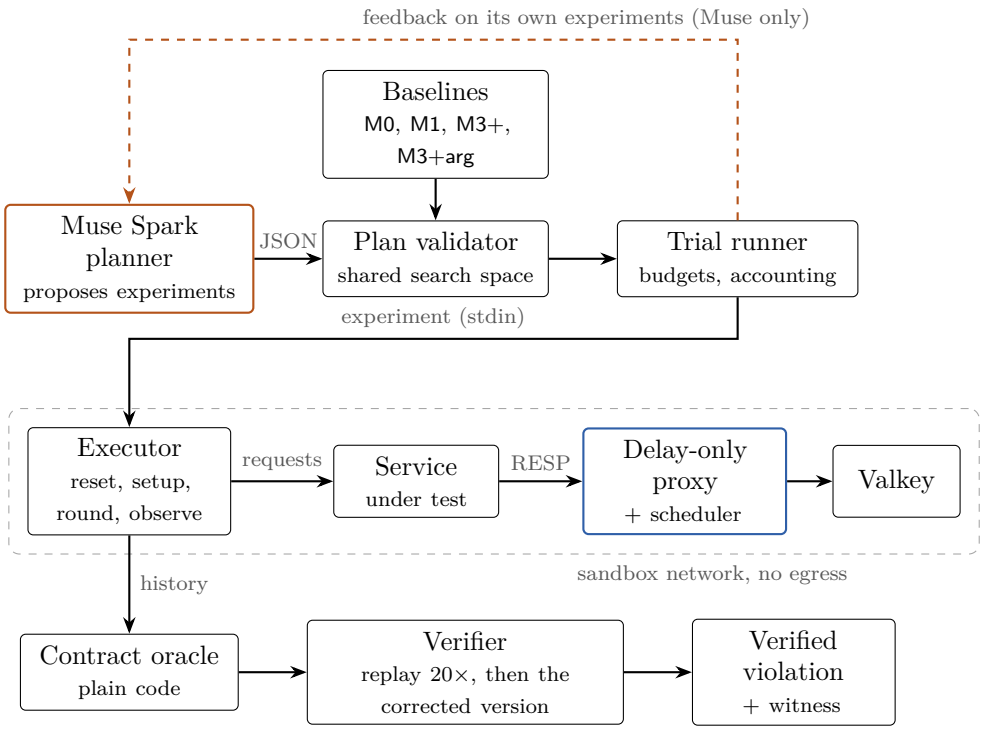


Figure 1: Components of Mylonite. The planner sees the catalog, contract, source and feedback, and emits validated experiments. Execution, scheduling, verdicts and verification are deterministic code shared by every method. The only other source of non-determinism is the stress baseline, which runs without schedule control.

legal and then enumerates the rest. `replay` follows a recorded decision list exactly and fails on divergence. A trace hash covers decisions, attributions, commands and digests, but not timestamps.

4.2 Contract oracle

Each operation is recorded as an invocation followed by `ok`, `fail` or `info`. A non-OK status counts as `info` (it may or may not have taken effect) unless the contract declares it to have no effect. Invariants are evaluated over the `ok` operations. For `info` operations, an invariant passes if some subset of them taking effect satisfies it. Invariant kinds are code with tests; they are never built from model text.

4.3 Verification

A `VIOLATION` is *verified* only if two conditions hold. First, its decision list replays 20 out of 20 times from fresh resets with identical trace hashes. Second, the same workload, explored on the corrected version of the service, yields no violation. That exploration is exhaustive when the workload has at most 5,000 schedules. Otherwise it covers the first 5,000 depth-first schedules plus 2,000 random ones, and the result is reported as *verified (bounded)*. Verification runs are not charged to any method.

4.4 The Muse planner

The only model Mylonite calls is `muse-spark-1.3` [22], in Standard mode, with `reasoning_effort` set to medium, the default temperature, and strict JSON-schema structured output.

What the model sees. One function builds each request from a declared view. The view contains the catalog, value domains, limits and observation; the contract's correctness conditions without their provenance fields; and the declared source files. The source files are shown under display names and normalized by parsing and unparsing, which removes comments and every bare string statement. From round 2 on, the view adds feedback on the model's own earlier experiments, drawn from an allow-list of fields.

Contamination audit. Every request is audited before it is sent and refused on a match. The audit looks for search artifacts (schedule labels, digests, verdict words, result keys); test-case labels, harness paths and fixture identifiers; and any identifier or line found only in the corrected version. Noninterference tests check three properties: changing anything outside the view leaves the request byte-for-byte unchanged, every detector fires on a planted positive, and a contaminated request is refused before any model call.

What the model returns. Each round, the model returns up to 3 experiments. Each one names a setup, the concurrent calls with arguments, a *schedule hint* (a list of store commands, each attributed to a concurrent call, in the order they should run), and its expected effect. The schema is generated from the catalog. The hint's command vocabulary is the proxy's full gated vocabulary, identical for every target, so the schema reveals nothing about which commands a target uses. A validator checks catalog membership, argument types and limits; it is the same validator that defines the baselines' search space. Each valid experiment runs hint-first, then depth-first, up to $K = 50$ schedules. A trial has at most 5 rounds.

Record and replay. Every request and response is logged with the model identifier, parameters, token counts and latency. A recorded trial can be re-executed without calling the API. The replay refuses if any request differs from the recording. Replay costs nothing, which makes it suitable for demonstrations.

4.5 Sandbox and accounting

The service, the store and an executor container (which runs the proxy, scheduler, executor and oracle) share one internal network with no egress and no published ports. The host orchestrator alone holds network access and the API key. It talks to the executor only over standard input and output. Python targets, including the held-out bases, implement a small harness interface (a connection function and one function per catalog operation) and run inside the executor container. Only D1, the OpenTelemetry cart, runs as a separate, unmodified service container. Every execution, including baseline profiling runs, goes through one trial runner, which counts it against the shared budget. *Method-seconds* are trial wall-clock time minus measured harness overhead and verification time. They include model latency and the method's own computation. Dollars are model tokens at list price, as recorded from the usage the API returned, plus container time at a fixed nominal rate.

Table 1: Methods compared. All share the catalog, domains, validator, executor, proxy, oracle, budgets and verifier; only workload choice and scheduling policy differ.

ID	Method	Workload choice	Schedule
M0	Stress	M1's random draw, run with no schedule control; a hit is reproduced by depth-first exploration under the scheduler, and every execution counts	real concurrency
M1	Random	per execution, 0–6 setup and 2–4 concurrent calls, operations and arguments uniform	`random(seed)`
M3+	Conflict-directed systematic	greybox breadth-first setup search; conflicting pairs explored, contract-relevant first	`exhaustive`, $\leq K$
M3+arg	M3+, argument-aware	as M3+, then the same calls with other argument values	`exhaustive`, $\leq K$
M4	Muse Spark	$\leq$ 5 rounds, $\leq$ 3 experiments each, reading the source	`hint`→`exhaustive`, $\leq K$
A2	Muse, no source	as M4, with the source removed from the view	`hint`→`exhaustive`, $\leq K$

5 Baselines and Ablation

Table 1 lists the methods. Every method draws from the same catalog, domains and validator, and runs under the same budgets: $B_{\text{exec}} = 1{,}000$ search executions, 900 method-seconds, and at most $K = 50$ schedules per explored workload. M3+ is the strongest non-LLM baseline we could build for this bug class. It profiles every catalog call alone through the proxy to obtain its command sequence with key digests. It runs a breadth-first search over setup states, in which a call that shows new behavior and reaches a new store-state digest extends the frontier. It then explores every pair of calls whose profiles admit an unserializable interleaving (the two-transaction conflict-cycle condition). Pairs that conflict on a key read by the contract's observation come first. M3+arg was added after the pilot. Once M3+'s frontier is exhausted, it explores pairs that M3+ deferred only because they share a command signature with an explored pair. The analysis treats M3+ and M3+arg together as one systematic baseline, and the comparison less favorable to Muse counts (§6.2).

Planned but not run. The plan also listed PCT (M2), an ablation pairing Muse's workloads with random schedules (A1), and an interface-renaming ablation (A3). None was run. M2 and A1 were marked "if time allows". Without A1, the contribution of Muse's schedule hint was not tested on the held-out targets.

6 Evaluation Design

6.1 Research questions

RQ1 Can Muse, using the source and execution feedback, find real concurrency failures in fewer executions than random or systematic exploration, under the same budgets and contracts?

RQ2 Where does any difference come from? The plan asked whether it comes from the workload

or from the schedule hint. We tested only whether reading the source matters, by comparing M4 with A2, which has no source.

RQ3 Does any advantage survive when model latency and cost are counted?

RQ4 Is every reported failure real? We measure replay reliability, corrected versions and negative controls.

A negative answer to RQ1 was declared in advance to be a valid result. As first written, RQ1 asked whether Muse finds failures "more efficiently". After the pilot, and before the freeze, the claim was re-scoped: execution efficiency (E_1) is reported separately from wall-clock time and cost, with no claim that Muse wins overall. A fifth question, on whether Muse reduces the effort of writing target specifications, was planned but not measured.

6.2 Frozen analysis

The analysis rules were frozen with the method (§6.3) and implemented as code with tests. The development results were known when these rules were written. Each rule is either the standard default or the choice less favorable to Muse.

- **Outcomes.** On each metric, a trial is *found* (at its cost), *failed* (its plan was exhausted, the time budget ran out, or a Muse trial errored) or *censored* (the execution budget ran out). An errored Muse trial is a failure, never a fresh draw.
- **A common scale.** For seconds and dollars, τ is the smallest cost spent by any trial censored at the execution budget in either group. Every cost above τ, and every non-finding, ranks as one tied failure. For E_1, $\tau = B_{\text{exec}}$, and censored values are $B_{\text{exec}} + 1$.
- **Test.** An exact two-sided Wilcoxon–Mann–Whitney test with mid-ranks, conditional on ties, with no approximation. The effect size is $\hat{A}_{12} = P(\text{Muse} < \text{baseline}) + \frac{1}{2}P(=)$; values above 0.5 favor Muse.
- **Stronger-baseline rule.** Muse is better than a group only if it is better than every member. The group's p for "better" is the largest member p, and is 1 unless every member's $\hat{A}_{12}$ favors Muse; this is an intersection–union test. "Worse" needs only one member, the smallest p among those Muse loses to. The headline group is *conventional* $= \{\mathsf{M0}, \mathsf{M1}, \mathsf{M3+}, \mathsf{M3+arg}\}$. The confirmatory groups are conventional and *systematic* $= \{\mathsf{M3+}, \mathsf{M3+arg}\}$.
- **Multiplicity.** The co-primary metrics are never combined. For each group, Holm's correction runs across the 6 cells (2 targets $\times$ 3 metrics), separately for "better" and "worse", at $\alpha = 0.05$. A non-significant cell is reported as "no significant difference", which is not a claim of equivalence.
- **Medians.** The median is the $\lceil n/2 \rceil$-th smallest value, and is "not reached" when more than half the trials failed. Intervals are percentile bootstraps with 10,000 resamples.
- **Break-even.** Each trial's cost to its first verified violation is a line in the cost per execution c: $E \cdot c$ plus its other cost (method-seconds minus execution seconds). Failed and censored Muse trials are infinite. A censored baseline trial takes the lower bound $(E+1)c$ plus what it had spent, so a baseline is never made to look slower than it is. The break-even c^* is where Muse's median line falls below the baseline's and stays below. The headline uses the baseline least favorable to Muse.
- **Scope.** Verdicts come only from held-out buggy targets. On a negative control, *any* `VIOLATION` is a false positive. Development and held-out results are never pooled.

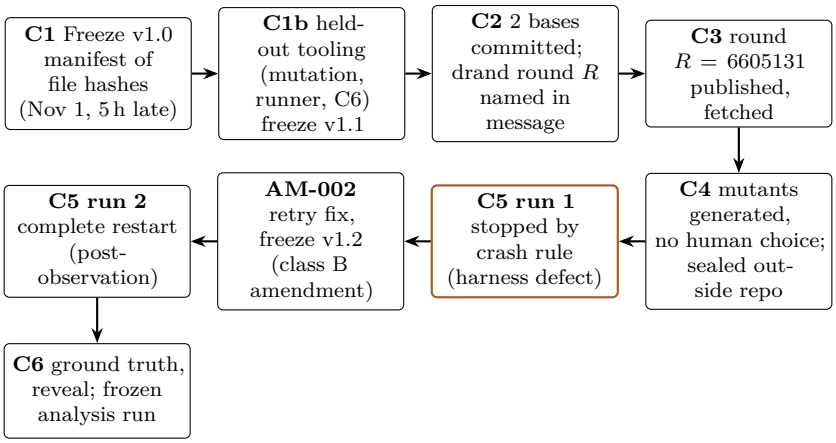


Figure 2: The held-out protocol as executed. Each box is a commit that the next step records. The restart is the only deviation from the planned order; the rule for combining the two runs was fixed in advance (§6.4).

6.3 Held-out protocol for a single developer

One developer was responsible for the prompt, the methods, the analysis and the base services, working with an AI coding assistant that wrote most of the code and documents, including the bases. We therefore do not claim independence. The protocol aims at two weaker properties that can be defended. First, the author's knowledge cannot choose which bugs exist or where they are. Second, the order of every step can be checked afterwards. The commit order can be checked from the repository now. The salted commitments can be checked only once the sealed directory and its salts are published, which has not yet happened. Figure 2 shows the steps; each is a commit that the next step records.

Freeze (C1). The prompt, schemas, methods, budgets, verification, analysis code and composition rules were frozen as a manifest of the SHA-256 hashes of 142 tracked files, tagged `freeze-v1.0`. The files are the method and harness code, tests, fixtures, target definitions, container images, configuration and rule documents. The held-out runner refuses to start unless the manifest check passes. Narrative Markdown and new experiment directories are exempt from the check. The check requires that every other listed file matches, that no unexpected file has been added, that the amendment log has only been appended to, and that every version after v1.0 has an entry in that log. The freeze was due at 23:59 ET on 31 October and was committed 5 hours late, on 1 November, with its real time. We record this as a deviation. Version 1.1 added the held-out tooling before any base existed. Version 1.2 is described in §6.4.

Bases (C2). Two base services were written after the freeze to specifications fixed beforehand. Base α handles event seat reservations: at least 8 catalog operations, and a precondition depth of at least 2 (an event must exist and be on sale before a reservation can succeed). Base β handles background-job leases: 6–8 operations, and a precondition depth of 1. Each base is deterministic Python using redis-py with one connection per client. Each has a correct contract written before any mutant existed. Each passed serial-validity, pass-through, contamination-audit and determinism checks, and has sites of at least 3 mutation-operator kinds, with at least 3 sites in each stratum. A site is *guarded* if it lies inside a branch that depends on a store read, and *unguarded* otherwise. The C2 commit message names drand round R as a number. The rule, applied mechanically, chose

the first mainnet round scheduled at least 6 hours and 5 minutes after the round was computed. The 5-minute margin, which we disclose, selected a round a few later than the protocol's 6-hour minimum. Round $R = 6605131$ was scheduled 6 hours 5 minutes after the commit. The first round named is binding. A FreeTSA RFC 3161 timestamp [2] was obtained for the C2 commitment.

Mutation (C3–C4). The frozen script implements four store-level operators: an atomic update becomes read, compute, write (O1); an optimistic transaction loses its `WATCH` (O2, narrowed from the plan, which also allowed watching the wrong key, an error the proxy cannot expose); a `MULTI/EXEC` block becomes non-transactional (O3); and a compare-and-delete lock release becomes an unconditional `DEL` (O5). It disclosed before C2 that two planned operators were not implemented. The seed is round R's randomness. Its first bit assigns the guarded stratum to one base and the unguarded stratum to the other. Within a base and stratum, a hash of the seed selects the first site, and later sites follow in cyclic order until a mutant passes a *serial-equivalence filter*. The filter requires identical outcomes on 150 random serial workloads and on every setup on the base's M3+ frontier followed by each catalog call. A mutant that differs serially is not a concurrency defect. The filter ran at C2, and its results are bound by the C2 commitment, so no discard can be claimed after R is known. Generated files were sealed outside the repository, and only their manifest was committed. No human opened a generated file before the results were committed.

Evaluation and reveal (C5–C6). The runner prints progress counts only. Under its crash rule, a trial that ends in a harness error is re-run once with the same seed, and both outcomes are reported. A mutant is *shown buggy* if any method produced a verified violation on it. A mutant not shown buggy would have gone to a pre-registered extended search, and would have been excluded for all methods alike only if that search also failed; this is the only exclusion rule. Each base serves as the negative control for its own mutant and as that mutant's corrected version.

Scope actually used. The plan called for 6 mutants from 8 bases, a weak-contract control, an equivalent mutant, multiplexed clients, a guaranteed shallow stratum and style-only decoy edits. Under time pressure, the developer fixed a minimum scope before C2: 2 mutants, with corrected negative controls and 20 trials per method per target. Each difference is a deviation, disclosed in §9.1.

6.4 Run validity, stop rules and the restart

Every model request must pass the audit, and every hit must be verified. Breaches of the thresholds for inconclusive executions and for soundness problems are flagged beside the verdicts, never used to exclude results. A run stops at once on a contamination refusal, a soundness problem, any violation on a base, or repeated executor failures. Reaching the model cost cap of $30 stops only the model trials. Restarting after a fix is a "class B" amendment, which restarts the evaluation from scratch. Under the pre-registered combination rule, the aborted run stays primary for every trial it completed, and the restart is reported beside it, labeled *post-observation*.

The first held-out run stopped when a crash-rule re-run failed with the same error: a connection reset had escaped the model client's retry rule. Amendment AM-002 (freeze v1.2) routed the model client's transport errors to the frozen retry rule. The second run completed without a stop. The primary analysis therefore uses run 1 for all 120 `alpha_m` trials and for seeds 1–10 of `beta_m`, and run 2 for seeds 11–20 of `beta_m` and all 200 control trials. Two run-1 trials ended in a model-client error, M4 seed 8 and A2 seed 10 on `beta_m`. Both count as failures, the reading less favorable to Muse.

Table 2: Development results: exploratory, known before the freeze, and never pooled with the held-out results. Each cell gives the median E_1 over $n = 5$ trials (seeds 1–5), then the median method-seconds per trial; "cens." means censored at $B_{\text{exec}} = 1{,}000$ in every trial. M0 ran on D2 and D3 in a later session, and M3+ there gave the same median (8 and 211).

Target	M0	M1	M3+	M4	A2
D2 counter (lost update)	9; 0.07 s	5; 0.06 s	8; 0.05 s	1; 14.2 s	1; 16.3 s
D3 inventory (check-then-act)	cens.	cens.	211; 1.7 s	1; 12.6 s	71; 31.0 s
D1 OpenTelemetry cart (#3824)	7; 0.05 s	18; 0.14 s	23; 0.12 s	not run	not run

7 Development Results

The development targets were used to build, tune and pilot the method. Their results were known when the rules were written, so they make no generalization claim (Table 2).

Fixtures. We wrote two Python fixtures. D2 is the counter of Listing 1. D3 is an inventory service with 8 operations, whose check-then-act bug in `purchase` is reachable only after `create`, `restock` and `publish`, and only when two purchases each fit the stock but together exceed it. Before any method ran, we computed that a random workload reaches D3's failure with probability about 2.6×10^{-6} per execution. Random search was therefore expected to be censored, and it was. In the pilot (Table 2), M4 found each bug on its first execution in every trial, while the best conventional medians were 5 executions (D2) and 211 (D3). A conventional method still won on wall-clock time on every development target: M4's 10 calls had a median latency of 12.6 s. The pilot spent $0.15 on 10 model calls, which used 18,590 prompt tokens (8,757 of them served from the cache) and 32,468 completion tokens. Listing 2 in the appendix shows Muse's recorded first experiment on D2. It proposes two awards to the same member, with the hint `GET`, `GET`, `SET`, `SET`. On the first execution, the balance ended at 2 rather than 3. The schedule replayed 20 of 20 times with an identical trace hash, and all 6 schedules of the same workload passed on the corrected version.

What the development ablation suggested. On D3, A2 found the bug in every trial, but needed a median of 71 executions. Its reasoning output was about 2.6 times longer than M4's. With the source, 5 of 5 first experiments used the right setup order; without it, 2 of 5 did. The other 3 published the product before restocking it, which wasted 64 to 107 executions until feedback repaired the order in round 2. All 5 A2 trials still named the right bug kind on `purchase`. That may be because the interface, through its operation names and contract text, names the domain plainly; no renaming ablation was run to test this. Two readings were recorded at the time, both as interpretations rather than measurements. The pilot noted that Muse appeared to read the bug out of the source, so the search side was barely exercised. The ablation added that, on D3, the source seemed to help M4 with the setup order rather than with spotting the bug. The pilot cannot separate reading code from planning. Moreover, both fixtures are textbook patterns written by the same author who wrote the prompt. These limits are why the held-out protocol exists.

The OpenTelemetry cart. We built the C# cart service from source at tag `3.1.0` and reproduced issue #3824 under schedule control. Every first violation replayed 20 of 20 times. The cart built with the fix proposed in PR #3825 produced no violation under exhaustive exploration of the same workload; its schedules either passed or were inconclusive. Without the proxy, two

simultaneous `AddItem` calls lost an update in 942 of 1,000 rounds on 28 October and in 927 of 1,000 on 1 November, against 0 of 1,000 for the fixed build. Stress testing found the bug in a median of 7 executions, at 5–9 ms each. We did not run Muse on this target, for three reasons. Its source view parses Python only, so the frozen M4 cannot read the C# service. The bug is public, so any result would carry a memorization caveat. And by our estimate, even if Muse found the bug on its first execution, it would need executions 230 to 420 times more expensive than they are to break even on time. D1 was labeled a case where the baselines win on 29 October, before the freeze. The reasons above come from a re-check run after the freeze, and the break-even is an estimate, since Muse never ran on D1.

8 Held-out Results

Both mutants used operator O1 and were shown buggy by verified witnesses from the evaluation itself, so nothing was excluded. In `alpha_m`, at a guarded site in `reserve`, an `HINCRBY` became `HGET` followed by `HSET`. In `beta_m`, at an unguarded site in `discard`, a `DECRBY` became `GET` followed by `SET`. Model spend was $25.01 over both runs, under the $30 cap.

RQ1: executions. Table 3 gives the confirmatory verdicts and Table 4 the per-method results. M4 found the bug in 20 of 20 trials on `alpha_m` and 19 of 20 on `beta_m`; the one failure is the errored run-1 trial, counted as a failure. 35 of its 39 verified trials found the bug at execution 1. On `beta_m`, the 4 remaining finds came at executions 38 to 88. Against the strongest baseline, M4 is better on E_1 on both targets: medians of 1 against 764 and 1 against 97. In the primary data, stress and random search never found `alpha_m`, which needs a two-step precondition; in run 2, stress verified it once. `beta_m`, at an unguarded site, needed no setup at all. All 20 primary M3+ witnesses, and 13 of M4's, started from an empty store. M3+ and M3+arg had identical E_1 on all 20 seeds of both targets, so M3+arg never exhausted M3+'s frontier.

RQ3: time and money. M4 is worse on method-seconds and on dollars on both targets. Its median find took 41.8 s and 59.9 s, against 5.8–6.5 s and 1.0 s for systematic search. Its median dollars per find were $0.033 and $0.047, against under $0.001. M4 made one model call in every `alpha_m` trial, and one or two in every completed `beta_m` trial. Figure 3 shows the trade-off.

Break-even. One execution costs about 9 ms on both mutants (medians 0.0090 s and 0.0091 s). By the frozen rule, M4 would be faster than every conventional method only if one execution cost more than 0.055 s on `alpha_m` (6 times the measured cost) and 0.62 s on `beta_m` (about 68 times). For dollars, the corresponding thresholds are 3.9 and 44 compute-seconds per execution, roughly 430 and 4,800 times the measured cost. A2 breaks even on time only above 0.43 s per execution on `alpha_m`. On `beta_m` it never breaks even, because it needs more executions than M3+.

RQ2: the source. Without the source, Muse was not better than the conventional group, or than systematic search, on any metric. In secondary comparisons, it was better on `alpha_m`'s E_1 than stress and random search taken alone (Holm $p = 7.7 \times 10^{-5}$ for each), since neither of them found that bug. On `alpha_m`, A2's median E_1 (401) was below M3+'s (764), but it found the bug in fewer trials (13 against 17), and the difference is not significant ($p = 0.84$). On `beta_m` it was worse ($p = 0.015$). In the secondary comparison (M4 against A2), M4 was better on E_1 on both targets (Holm $p = 8.7 \times 10^{-11}$ and 5.3×10^{-7}). It was also better on seconds and dollars on `alpha_m`. On `beta_m` there was no significant difference on those two metrics ($p = 0.22$ and 0.11).

Table 3: Confirmatory verdicts of Muse against the conventional group {M0, M1, M3+, M3+arg} (primary combination). Holm-adjusted exact two-sided Mann–Whitney p-values. The systematic group {M3+, M3+arg} gives the same verdict in every cell.

Condition	Metric	`alpha_m` (guarded)	`beta_m` (unguarded)
M4 (with source)	E_1	**better**, $p = 8.7 \times 10^{-11}$	**better**, $p = 2.0 \times 10^{-5}$
	method-seconds	worse, $p = 1.0 \times 10^{-7}$	worse, $p = 8.7 \times 10^{-11}$
	dollars	worse, $p = 1.0 \times 10^{-7}$	worse, $p = 8.7 \times 10^{-11}$
A2 (no source)	E_1	no significant difference, $p = 0.84$	worse, $p = 0.015$
	method-seconds	worse, $p = 1.0 \times 10^{-7}$	worse, $p = 8.7 \times 10^{-11}$
	dollars	worse, $p = 1.0 \times 10^{-7}$	worse, $p = 8.7 \times 10^{-11}$

Table 4: Per-method results, 20 trials each. Verified trials; median E_1 with 95% bootstrap interval ("n.r.": not reached, more than half failed); medians of method-seconds and dollars to the first verified violation. A2's dollar medians are rounded to two places. The results summary reports 0.124 and 0.072; an independent recomputation from the trial files gives 0.123 and 0.071.

Target	Method	Verified	Median E_1 [95% CI]	Seconds	Dollars
`alpha_m`	M0	0	n.r.	n.r.	n.r.
	M1	0	n.r.	n.r.	n.r.
	M3+	17	764 [546, 856]	6.5	< 0.001
	M3+arg	17	764 [546, 856]	5.8	< 0.001
	M4	**20**	**1 [1, 1]**	41.8	0.033
	A2	13	401 [254, n.r.]	159.5	0.12
`beta_m`	M0	20	205 [131, 300]	2.0	< 0.001
	M1	18	292 [131, 582]	3.9	< 0.001
	M3+	20	97 [82, 127]	1.0	< 0.001
	M3+arg	20	97 [82, 127]	1.0	< 0.001
	M4	**19**	**1 [1, 1]**	59.9	0.047
	A2	19	137 [137, 201]	68.1	0.07

RQ4: false positives and verification. In 200 control trials on the two bases (M0, M1, M3+, M3+arg and M4, 20 each), no source produced a `VIOLATION`: not scheduled executions, stress runs, serial profiles, or the corrected-version checks during witness verification. Every one of the 183 verified held-out witnesses replayed 20 of 20 times. The corrected-version check was bounded rather than exhaustive for 42 of them, as the frozen rule allows: 26 of A2's 32, 9 of M1's 18, 6 of M0's 20, and 1 of M4's 39.

Sensitivity. Run 2 alone is complete, single-version and post-observation, and it gives the same confirmatory verdict in every cell. Two secondary cells change: M4 against A2 on `beta_m` becomes "better" on seconds and dollars, because run 2 has no errored trial and M4 finds `beta_m` in 20 of 20. Taking A2 seed 10 from run 2 changes no verdict. Among trials completed in both runs, the seeded methods M1, M3+ and M3+arg gave the same status and E_1 on every trial, and M4 had the same E_1 in 27 of the 29 trials that both runs verified.

Recomputation. A separate reviewer recomputed every result without importing the analysis code, using a different exact algorithm for the Mann–Whitney test. The reviewer was an AI

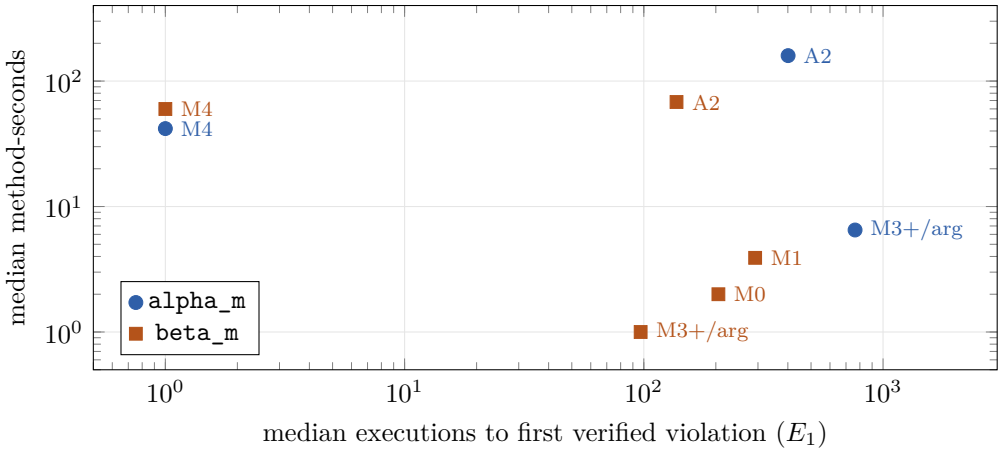


Figure 3: Executions against seconds on the two held-out bugs, medians of 20 trials. Compared with systematic search, Muse with source (M4) needs 97 to 764 times fewer executions but takes 6 to 60 times longer. On `alpha_m`, M0 and M1 never found the bug and are not plotted; M3+arg took 5.8 s there.

agent, not a person. It reproduced the 12 verified counts and medians, all 48 comparisons, the 24 confirmatory verdicts with their Holm values, all 32 break-evens and both sensitivity variants. It found two places where the frozen code differs from the frozen text. In one, a time-limited baseline gets a finite lower bound. In the other, the false-positive scan covers only the records the analysis selected. Neither changes any result, and neither was fixed, since a fix after observation would itself be an amendment. A scan of all 882 records in both runs found no corrected-version violation.

9 Discussion

What is shown. On these two mutants, Muse with source access needed far fewer executions than every conventional baseline, including the systematic search designed for this bug class. At the cost of an execution in our setting, it was nonetheless slower and more expensive. The headline is therefore a trade-off, not a win. Whether the model is worth calling depends on how expensive an execution is.

The small p-values measure repeatability, not breadth. M4's trials are near-replicates. Every round-1 prompt has 3,078 tokens on `alpha_m` and 2,461 on `beta_m`. In 12 of the 20 primary `alpha_m` trials, the provider served 3,057 of those tokens from its prompt cache, and part of them in one more trial. In run 1, 19 of the 20 `alpha_m` witnesses used the same workload. A p-value near 10^{-11} thus says that M4 reliably repeats one good plan on each target. It does not say that M4 would win on a different defect. Twenty seeds of one bug measure reliability; generalization needs more bugs.

Why the source helps: a hypothesis. We hypothesize that M4 finds each bug by reading the mutated line. In each mutant, the mutated lines are the only read-then-write of a counter outside a `WATCH` transaction; every other update of that counter is atomic. The ablation is consistent with it: without the source, Muse was not better than the conventional group, although it did beat stress and random search on `alpha_m`. It is still a hypothesis, not a measurement. Both mutants come

from one operator, there were no decoy edits, and the planner's instructions name `lost_update` and `check_then_act` among the hypothesis kinds, which is the author's prior.

When the model would pay off. On time, the break-even is about 6 and 68 times our measured execution cost; on dollars, roughly 430 and 4,800 times. Targets whose executions are genuinely expensive, such as remote environments or slow setup paths, are where Muse could win on time, if its execution advantage carries over. More containers alone are not enough: our multi-container D1 target still cost 5–9 ms per execution. We did not test such targets.

9.1 Threats to validity

Internal validity. A single developer directed the work. An AI coding assistant wrote most of the code and documents, including the held-out base services, and it proposed the analysis rules. Several rules, including the headline conventional-group rule, were adopted as the assistant's proposals, in force unless the developer vetoed them, rather than by explicit approval. The bases were written knowing the operators, so that losing atomicity could violate their contracts. The author expected each base's O3 site not to violate its contract, and that site was also each base's only route to a third operator kind. Each stratum had exactly 3 live sites because of one style choice. One site was dead, and some interleavings are covered by no contract clause. Before C2, a safety check ran 41,378 executions on the unmutated bases, covering exhausted systematic pairs, random groups and stress runs, and found no violation. A redis-py retry on timeout could in principle run a held command twice. The safety check recorded no forced decision, and rounds take milliseconds, so this is unlikely to have mattered. Blindness was procedural: with the beacon value and the frozen script, the developer could have computed the mutated site without opening a file. The freeze was 5 hours late. Its analysis rules were written after the development results were known, though each is standard or less favorable to Muse. The mutation script, runner and ground-truth script were written after the freeze, in version 1.1, before any base existed. Commit times are self-asserted, and the freeze manifests are not externally anchored. The FreeTSA timestamp of the bases predates the beacon round by about 6 hours, but its certificate chain was not verified. At the reveal, the beacon response fetched again was byte-identical to the original, and its fields matched a second drand endpoint, but its BLS signature was not verified. One restart supplied 60 `beta_m` trials and all 200 controls, all post-observation. The analysis driver was written after outcomes were observed. It contains no rule of its own and was committed before its first run.

Construct validity. E_1 counts executions, not effort. Method-seconds include model latency at the time of the runs. The prompt cache is shared across trials, so dollar costs depend on trial order. The dollar verdict is "worse" either way. Some verifications were bounded rather than exhaustive. Nine M4 control trials ran past the 900-second budget, because a model call already in flight runs to completion; this affects controls only.

External validity. There are two mutants, both lost updates from one operator, in small Python services with one connection per client, written by the same developer who ran the evaluation. There were no decoy edits and no guaranteed shallow stratum. Both omissions favor a model that reads source. Undisclosed before the reveal: the generator rewrites each mutant file through Python's `ast.unparse`, so a mutant's formatting differs from its hand-written base. Formatting is uniform within a mutant, and no method sees both files in one trial. The model view also normalizes every file through the same parse-and-unparse step, so neither Muse condition could see the formatting difference. Executions were fast and local. One model version was tested.

Deviations from the plan. The deviations recorded in the project are:

- **Scope:** 2 mutants instead of 6, from 2 bases instead of 8; no weak-contract control and no equivalent mutant; no multiplexed clients; no decoys; and no shallow stratum.
- **Operators:** two operators not implemented, and O2 narrowed.
- **Methods and questions:** PCT, A1 and A3 not run; RQ1 re-scoped after the pilot.
- **Freeze and schedule:** a late freeze, with the held-out set not sealed by the freeze gate. The gate's fallback was not taken, by the developer's decision. A later schedule gate was replaced by the minimum-scope schedule, and the held-out tooling was written after the freeze.
- **Procedure:** a drand margin that selected a slightly later round; one non-append edit to the amendment log (a placeholder line replaced when the first amendment was added); and one restart.
- **Reporting:** after observation, the reading of one errored trial was revised to the reading less favorable to Muse. The results summary contains errata, recorded in the decision log, including a prompt-token count stated for both targets and a sentence that presents the source-reading interpretation as shown. It also gives 13 rather than 12 for the cache count above.

10 Ethics and Responsible Use

Mylonite is a defensive testing tool. Every target ran in a local, disposable sandbox with no egress and synthetic data. We did not test any live third-party system. The one third-party target, the OpenTelemetry cart, was built from public source and run locally, and its bug had already been reported publicly. The model receives only an allow-listed view of the source, with comments removed. It never writes code that runs; it only chooses among catalog operations. The design keeps credentials out of prompts, traces and the repository, and a full-history secret scan of the repository found no API key. At the time of writing, the sealed held-out sources were not yet published; their publication is the owner's decision.

Use of AI assistance. An AI coding assistant (Claude Code, by Anthropic) wrote most of Mylonite's code and documents under the developer's direction, including the held-out base services. It also proposed the analysis rules. Decisions were either approved by the developer or adopted as the assistant's proposals, in force unless the developer vetoed them; several rules, including the headline group rule, were adopted that way. Every commit is co-authored by the assistant. Reviews described here as independent were performed by separate AI agents, not by people. This manuscript was also drafted by the assistant from the project's records, and a separate AI agent fact-checked it against them. The assistant is not part of Mylonite's runtime: Muse Spark is the only model Mylonite calls.

11 Conclusion

We built a harness that lets a language model choose which requests to race against a Redis-backed service. Deterministic code schedules the store commands, proves every bug with a replayable witness, and judges every outcome. We then evaluated the model's choice under a pre-registered, single-developer held-out protocol against strong non-AI baselines. On two held-out lost updates,

the model with source access found each bug in a median of one execution, against 764 and 97 for the best baseline. Because executions cost milliseconds and a model call costs tens of seconds, systematic search was still faster and cheaper. The model would be faster only if an execution cost about 6 to 68 times more than ours does, and cheaper only at roughly 430 to 4,800 times. Future work should test defects of other kinds (check-then-act, multi-key, locks), with decoy edits; services in other languages, with multiplexed clients; and targets whose executions are genuinely expensive.

Availability. The code, trial records, frozen manifests, analysis and its outputs are at `https://github.com/harthikrm/mylonite` (Apache-2.0) [19]. The freeze is tagged `freeze-v1.0`, and the held-out analysis is reproducible from the committed trial files. The raw run logs and the sealed held-out sources are not published, so the held-out trials themselves cannot yet be re-run from the repository. The development pilot can be replayed from its recorded model responses at no cost. The replay matched the original pilot on status, E_1, executions, witness and counters.

References

[1] Parosh Abdulla, Stavros Aronis, Bengt Jonsson, and Konstantinos Sagonas. Optimal dynamic partial order reduction. In *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2014.

[2] Carlisle Adams, Pat Cain, Denis Pinkas, and Robert Zuccherato. Internet X.509 public key infrastructure time-stamp protocol (TSP). RFC 3161, 2001.

[3] Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2011.

[4] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. Feral concurrency control: An empirical investigation of modern application integrity. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*, 2015.

[5] Sebastian Burckhardt, Chris Dern, Madanlal Musuvathi, and Roy Tan. Line-up: A complete and automatic linearizability checker. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2010.

[6] Sebastian Burckhardt, Pravesh Kothari, Madanlal Musuvathi, and Santosh Nagarakatte. A randomized scheduler with probabilistic guarantees of finding bugs. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2010.

[7] Cormac Flanagan and Patrice Godefroid. Dynamic partial-order reduction for model checking software. In *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2005.

[8] Fray developers. Fray MCP server documentation. `https://github.com/cmu-pasta/fray/blob/main/docs/mcp.md`. Accessed October 2026.

[9] Sishuai Gong, Dinglan Peng, Deniz Altınbüken, Pedro Fonseca, and Petros Maniatis. Snowcat: Efficient kernel concurrency testing using a learned coverage predictor. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[10] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 1990.

[11] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[12] Jepsen. Jepsen: Distributed systems safety research. https://github.com/jepsen-io/jepsen. Accessed October 2026.

[13] Kyle Kingsbury and Peter Alvaro. Elle: Inferring isolation anomalies from experimental observations. *Proceedings of the VLDB Endowment*, 2020.

[14] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.

[15] League of Entropy. drand: a distributed randomness beacon. https://drand.love. Mainnet round 6605131 used here.

[16] Ao Li, Byeongjee Kang, Vasudev Vikram, Isabella Laybourn, Samvid Dharanikota, Shrey Tiwari, and Rohan Padhye. Fray: An efficient general-purpose concurrency testing platform for the JVM. *Proceedings of the ACM on Programming Languages*, (OOPSLA), 2025.

[17] Ziyang Li, Saikat Dutta, and Mayur Naik. IRIS: LLM-assisted static analysis for detecting security vulnerabilities. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[18] Shan Lu, Soyeon Park, Eunsoo Seo, and Yuanyuan Zhou. Learning from mistakes: A comprehensive study on real world concurrency bug characteristics. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2008.

[19] Harthik Mallichetty. Mylonite: source code, trial data and analysis. https://github.com/harthikrm/mylonite, 2026. Apache-2.0. Method freeze tag freeze-v1.0.

[20] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[21] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. Large language model guided protocol fuzzing. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.

[22] Meta. Muse spark (muse-spark-1.3), via the Meta model API. https://dev.meta.ai/docs/models, 2026. Standard mode; structured output; reasoning_effort medium.

[23] Madanlal Musuvathi and Shaz Qadeer. Iterative context bounding for systematic testing of multithreaded programs. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2007.

[24] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gérard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtiu. Finding and reproducing heisenbugs in concurrent programs. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2008.

[25] OpenTelemetry Demo contributors. Cart service has no atomicity guarantees — concurrent AddItem calls lose updates, and checkout's EmptyCart can silently discard items added during checkout (issue #3824). https://github.com/open-telemetry/opentelemetry-demo/issues/3824. Opened 10 August 2026; still open when checked on 1 November 2026.

[26] Paul Thomson, Alastair F. Donaldson, and Adam Betts. Concurrency testing using controlled schedulers: An empirical study. *ACM Transactions on Parallel Computing*, 2016.

[27] Tengfei Tu, Xiaoyu Liu, Linhai Song, and Yiying Zhang. Understanding real-world concurrency bugs in Go. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.

[28] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2024.

[29] András Vargha and Harold D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.

[30] Todd Warszawski and Peter Bailis. ACIDRain: Concurrency-related attacks on database-backed web applications. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*, 2017.

[31] Cheng Wen, Mengda He, Bohao Wu, Zhiwu Xu, and Shengchao Qin. Controlled concurrency testing via periodical scheduling. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2022.

[32] Dylan Wolff, Zheng Shi, Gregory J. Duck, Umang Mathur, and Abhik Roychoudhury. Greybox fuzzing for concurrency testing. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[33] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2024.

[34] Xinhao Yuan, Junfeng Yang, and Ronghui Gu. Partial order aware concurrency sampling. In *Proceedings of the International Conference on Computer Aided Verification (CAV)*, 2018.

[35] Zhang, Păsăreanu, and Yang. Agent-guided thread scheduling for concurrency bug detection and reproduction. arXiv:2610.13421, 2026. Submitted 9 October 2026.

A Muse's Recorded First Experiment on D2

Listing 2 shows the first experiment of Muse's first response in pilot trial `d2-loyalty-points/M4/1` (28 October 2026). Its values are unchanged, but its keys are shown in a different order, and its `hypothesis_id` and the response's other fields are omitted. The witness that this experiment produced on execution 1 had the decision list `admit:0, exec:0, admit:1, exec:1, exec:0, exec:1`. The balance went from 0 to 2, against an allowed delta of 3, violating

`balance_matches_awards`. The witness replayed 20 of 20 times with identical trace hashes, and the corrected version passed all 6 schedules under exhaustive exploration.

Listing 2: Muse's first experiment, trial `d2-loyalty-points/M4/1` (`muse-spark-1.3`).

```
"setup": [],
"concurrent": [
  {"args": {"member": "m1", "points": 1}, "op": "award"},
  {"args": {"member": "m1", "points": 2}, "op": "award"}
],
"schedule_hint": [
  {"command": "GET", "op_index": 0},
  {"command": "GET", "op_index": 1},
  {"command": "SET", "op_index": 0},
  {"command": "SET", "op_index": 1}
],
"expected_effect": "Both awards GET 0 then SET 1 and 2; last writer wins so final
    balance is 1 or 2 instead of 3, violating balance_matches_awards"
```

B Provenance of the Held-out Evaluation

Step	Commit	What
C1	`da3ca57`	Method freeze v1.0 (tag `freeze-v1.0`)
C2	`50fe826`	Bases committed; round $R = 6605131$ named; FreeTSA timestamp `9d09957`
C4	`76264cf`	Mutants generated from round 6605131 and sealed
C5 run 1	`f82681e`	Stopped by the crash rule on a harness defect
AM-002, v1.2	`e19d818`, `3bb8957`	Model-client transport errors reach the frozen retry rule
C5 run 2	`8e249c3`	Complete restart under v1.2; post-observation
C6	`0cf4fb6`	Both mutants shown buggy; nothing excluded
Driver	`cac2654`	Analysis driver, committed before its first run

C Cost

Model spend was $25.01 over both held-out runs ($4.58 and $20.44) and $25.45 including development, as recorded from the usage the API returned. The held-out cap was $30. The original plan, with 10 held-out targets, had an owner-approved cap of about $100. Its estimate was about $63 for M4, $25 for A2 and $10 for the pilot, specification drafting and the demo.